Experiment No.: 7

Title: Preparing Software Design Document (SDD)

Batch: B3 **Roll No.: 16014225801** **Experiment No.: 7**

Aim: To prepare Software Design Document (SDD)

Resources needed: Internet Explorer, LaTex Editor

Theory

The SDD shows how your software will be structured to satisfy the requirements. It describes the software structure, software components, interfaces, and data necessary for the implementation phase. In a complete SDD, each requirement must be traceable to one or more components.

An SDD is a representation or model of the software system to be created. The model should provide the precise design information needed for planning, analysis, and implementation of the software system. It should represent a partitioning of the system into components and describe the important properties and relationships between them.

SDD Template:

1 INTRODUCTION

1.1 Design Overview

Give a description of the design approach, highlighting essential features that allow the design to meet the stated requirements.

1.2 Requirements Traceability Matrix

Provide a matrix showing where each feature identified in the SRS is supported by the design components.

	Component 1	*Component 2*	*Component 3*	*Component 4*
Requirement 1		*X*		*X*
Requirement 2	*X*	*X*	*X*	*X*
Requirement 3			*X*	
Requirement 4	*X*			*X*

2 SYSTEM ARCHITECTURAL DESIGN

2.1 Chosen System Architecture

Describe the system architectural design, identifying the major component groupings and the interfaces (both internal and external).

Make sure to identify any significant technical risks, and identify contingency plans for each.

2.2 Discussion of Alternative Designs

Discuss in a reasonable level of detail other design options explored, and the reasons for not choosing them.

2.3 System Interface Description

Describe the system interfaces in detail: O/S interface, files, networking, libraries, graphics libraries etc. (*Describe the user interface in section 4.)

3 DETAILED DESCRIPTION OF COMPONENTS

3.n Component-n

For each component, the following items should be described here as appropriate: responsibilities, constraints, composition, interactions, and resources. Use appropriate diagrams or other notation to describe your design.

4 USER INTERFACE DESIGN

In this section describe the design of the user interface in detail.

4.1 Description of the User Interface

4.1.1 Screen Images

Show the design of layout and menus for each screen.

4.1.2 Objects and Actions

Identify all the objects on each screen, and define the actions to be taken by each object for each event.

5. System Architecture

In this section high-level overview of how the functionality and responsibilities of the system were partitioned and then assigned to subsystems or components are provided.

Use cases from the SRS Document

Use Case Specification using template

Use Case ID:	**1**		
Use Case Name:			
Created By:		Last Updated By:	
Date Created:		Date Last Updated:	

Primary Actors:	
Secondary Actors:	
Description:	
Trigger:	
Preconditions:	
Postconditions:	
Normal Flow:	
Alternative Flows:	
Exceptions:	
Includes:	

Priority:	
Frequency of Use:	
Business Rules:	
Special Requirements:	
Open Issues	
Assumptions:	
Notes and Issues:	

6. Data flow specifications

DFD is created from the SRS document provided.

6.1 Level 0 DFD with description

6.2 Level 1 DFD with description

Procedure:

1. Prepare SDD document for chosen problem definition in LaTeX.

Results: Software Design Document (SDD) in given format

Questions:

1. Explain Architecture design patterns and styles with examples.

Answers

Architectural styles and design patterns provide structural blueprints for organizing software systems to achieve scalability, maintainability, and reliability. An architectural style defines a high-level family of system organization through components, connectors, and topological constraints, whereas an architectural design pattern offers a concrete, reusable solution to a recurring structural problem within a specific context. Common styles include the Layered (N-Tier) style, which partitions systems into hierarchical layers like Presentation, Business Logic, and Data Access (e.g., enterprise web apps and the OSI model), the Client-Server style (e.g., web browsers and servers), and the Pipe-and-Filter style, which routes data streams through sequential transformations (e.g., compilers and UNIX pipelines). Key architectural patterns include Model-View-Controller (MVC), which separates data, user interface, and control logic (e.g., Django, Spring); Microservices, which decomposes applications into independently deployable services (e.g., Netflix, Amazon); and the Event-Driven (Publish-Subscribe) pattern, which uses brokers for asynchronous messaging (e.g., Apache Kafka).

Conclusion:
Hence we performed the experiment.

Grade: AA / AB / BB / BC / CC / CD /DD

Signature of faculty in-charge with date

References:

Books:

1. Roger S. Pressman, Software Engineering: A practitioners Approach, 7th Edition, McGraw Hill, 2010.

2. Technical report on Guidelines for Documents Produced by Student Projects In Software Engineering based on IEEE standards

3. Timothy C. Lethbridge, Robert Laganiere " Object-Oriented Software Engineering – A practical software development using UML and Java", Second Edition, Tata McGraw-Hill, New Delhi,2004